

C 언어 EXPRESS



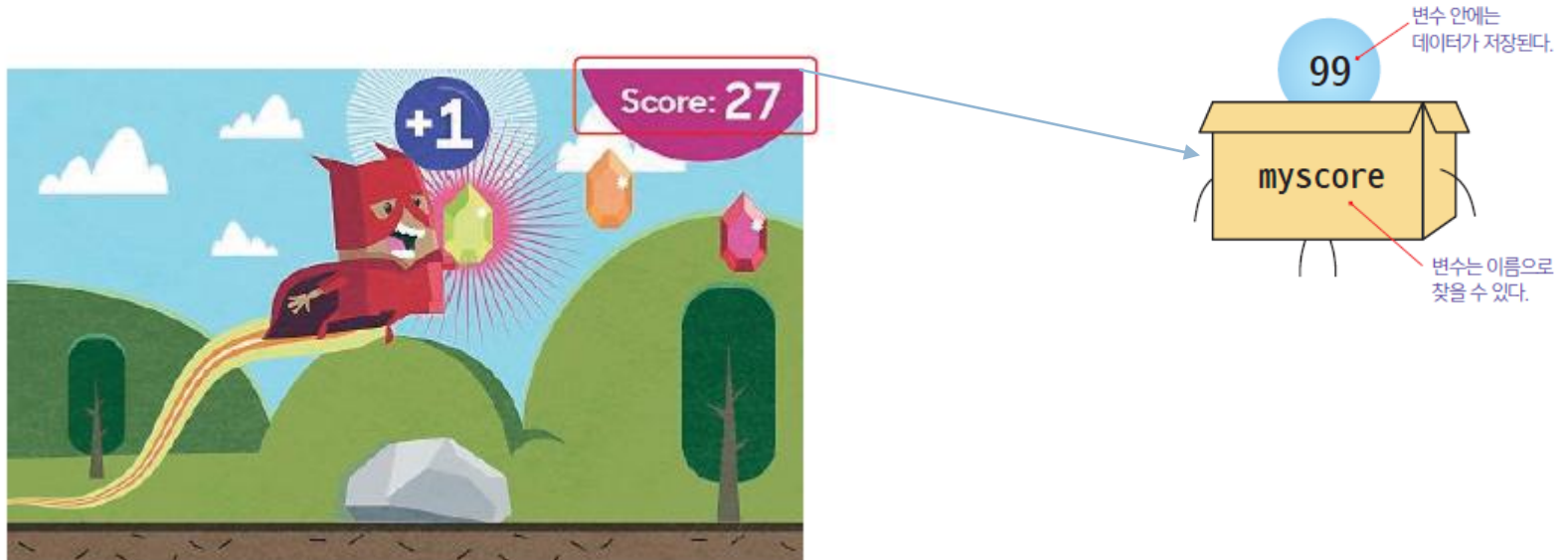
제 4장 변수와 자료형



변수

2

- 컴퓨터 프로그램은 값을 저장하기 위하여 변수(variable)를 사용한다.
- 변수는 게임에서 점수를 저장하는데 사용될 수 있고, 대형 마트에서 우리가 구입한 물건들의 가격을 저장할 수도 있다.

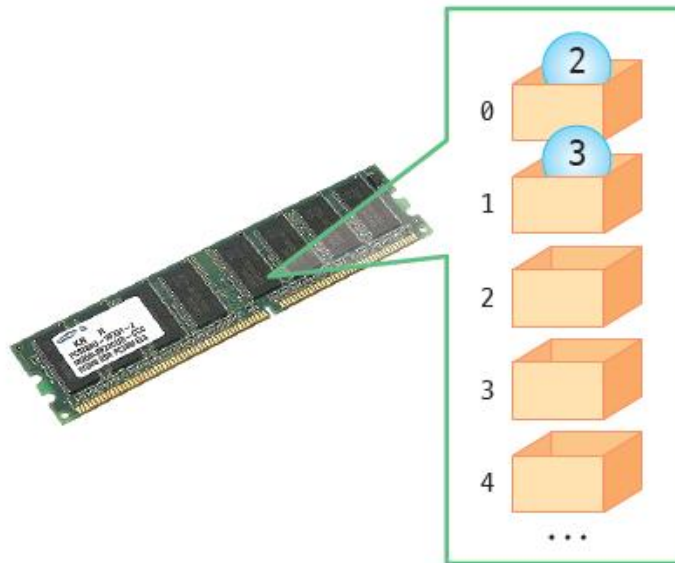




변수는 어디에 만들어지는가?

3

- 변수는 바로 메인 메모리(main memory)에 만들어진다.
- 우리는 변수 이름을 사용하여 메모리 공간을 사용하게 된다

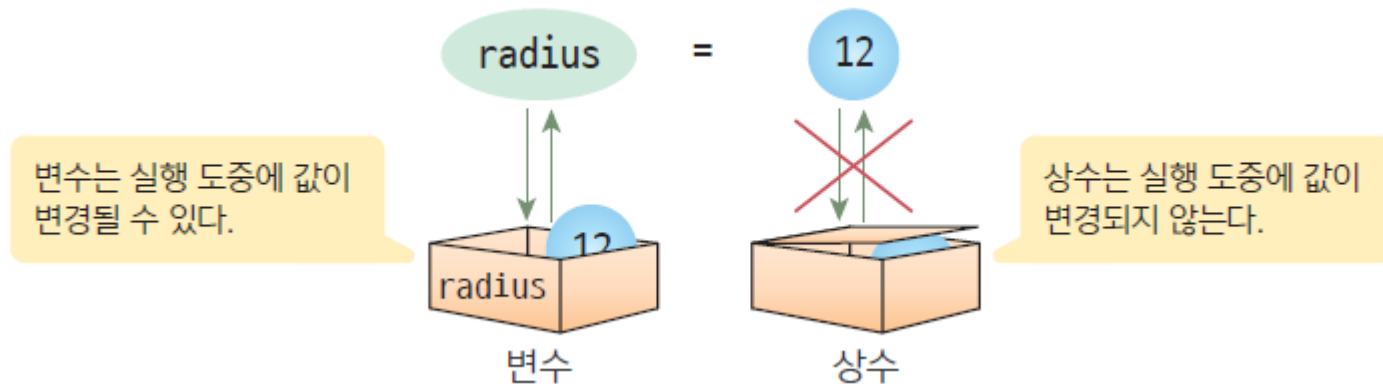




변수와 상수

4

- **변수(variable)**: 저장된 값의 변경이 가능한 공간
- **상수(constant)**: 저장된 값의 변경이 불가능한 공간
 - (예) 3.14, 100, 'A', "Hello World!"





예제: 변수와 상수

5

```
/* 원의 면적을 계산하는 프로그램 */ 변수
#include <stdio.h>
int main(void)
{
    float radius;
    float area;

    // 원의 반지름
    // 원의 면적

    printf("원의 면적을 입력하세요:");
    scanf("%f", &radius);

    area = 3.141592 * radius * radius;
    printf("원의 면적: %f \n", area);

    return 0;
}
```

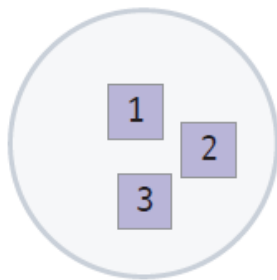
scanf() 오류가 발생하면 소스의 첫 부분에
#define _CRT_NO_SECURE_WARNINGS
을 넣어야함.



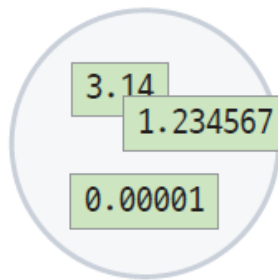
자료형

6

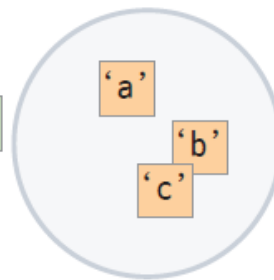
- 자료형(data type): 데이터의 타입(종류)
 - short, int, long: 정수형 데이터(100)
 - double, float: 부동소수점형 데이터(3.141592)
 - char: 문자형 데이터('A', 'a', '한')



정수형



부동 소수점형



문자형





다양한 자료형이 필요한 이유

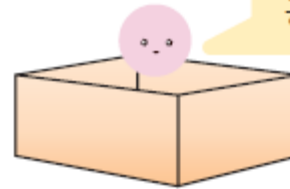
7

- 상자에 물건을 저장하는 것과 같다.

물건이 상자보다 크면
들어가지 않을 것이다.



물건이 상자보다 너무 작으면
공간이 낭비될 것이다.

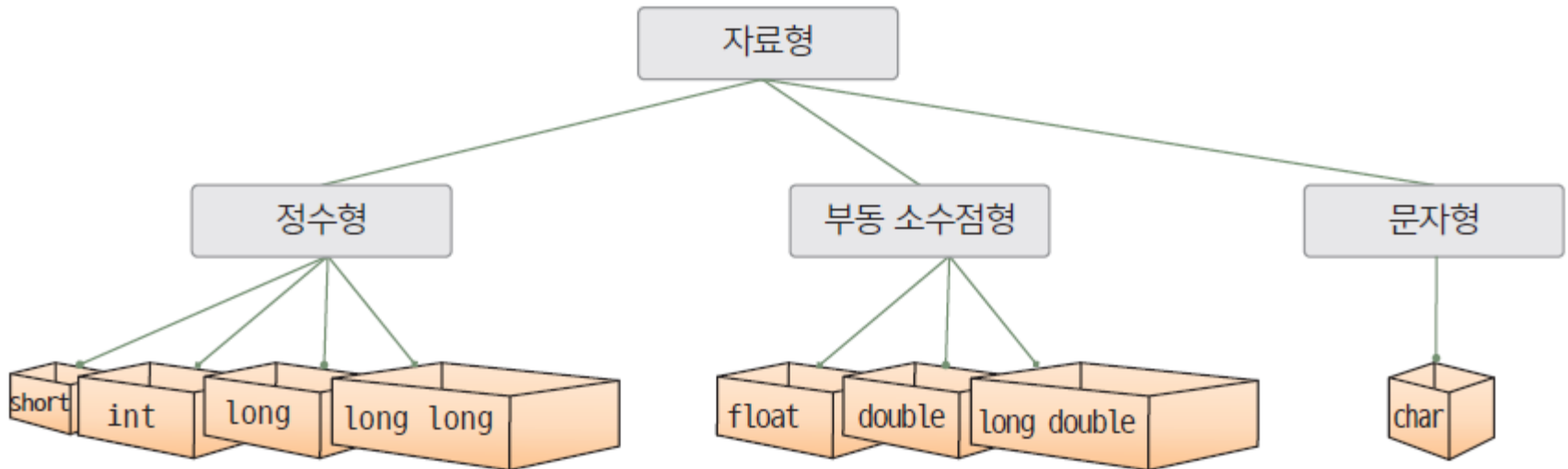




자료형의 분류

8

- 자료형을 크게 나누면 정수형(integer type), 부동소수점형(floating-point type), 문자형(character type)으로 나눌 수 있다.





예제: 자료형의 크기

9

- 자료형의 크기를 알아보려면 `sizeof` 연산자를 사용하면 된다. `sizeof`는 변수나 자료형의 크기를 바이트 단위로 반환하는 연산자이다.

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int x;
```

```
    printf("변수x의   크기: %d\n", sizeof(x));
```

```
    printf("char형의   크기: %d\n", sizeof(char));
```

```
    printf("int형의   크기: %d\n", sizeof(int));
```

```
    printf("short형의  크기: %d\n", sizeof(short));
```

```
    printf("long형의   크기: %d\n", sizeof(long));
```

```
    printf("float형의  크기: %d\n", sizeof(float));
```

```
    printf("double형의 크기: %d\n", sizeof(double));
```

```
    return 0;
```

```
}
```

변수x의	크기: 4
char형의	크기: 1
int형의	크기: 4
short형의	크기: 2
long형의	크기: 4
float형의	크기: 4
double형의	크기: 8



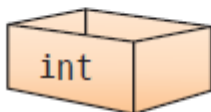
정수형

10

자료형		비트	범위
정수형	short	16비트	-32768~32767
	int	32비트	-2147483648~2147483647
	long		
	long long	64비트	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807



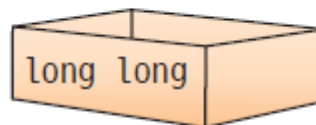
≤



≤



≤



16비트(2바이트)

≤

32비트(4바이트)

≤

32비트(4바이트)

≤

64비트(8바이트)



C에서는 왜 이렇게 많은 종류의 정수형이 있을까?

11

- 용도에 따라 프로그래머가 선택하여 사용할 수 있게 하려는 것이다.
- 비트수를 늘리면 정수의 범위는 확대시킬 수 있지만 메모리 공간을 더 많이 필요로 한다.



어떤 크기던지
돌릴 수 있습니다.
하지만 복잡하고
크기가 큼니다.



크기에 맞추어
선택하여
사용합니다.



정수형의 범위

12

약 -21억에서
+21억

- int형

$-2^{31}, \dots, -2, -1, 0, 1, 2, \dots, 2^{31} - 1$
(-2147483648 ~ +2147483647)

- short형

$-2^{15}, \dots, -2, -1, 0, 1, 2, \dots, 2^{15} - 1$
(-32768 ~ +32767)

- long형

- 보통 int형과 같음





예제

13

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    short year = 0;           // 0으로 초기화한다.
```

```
    int sale = 0;             // 0으로 초기화한다.
```

```
    long total_sale = 0;      // 0으로 초기화한다.
```

```
    long long large_value;    // 64비트 자료형
```

```
    year = 10;                // 약 3만2천을 넘지 않도록 주의
```

```
    sale = 2000000000;        // 약 21억을 넘지 않도록 주의
```

```
    total_sale = year * sale;  // 약 21억을 넘지 않도록 주의
```

```
    printf("total_sale = %d \n", total_sale);
```

```
    return 0;
```

```
}
```

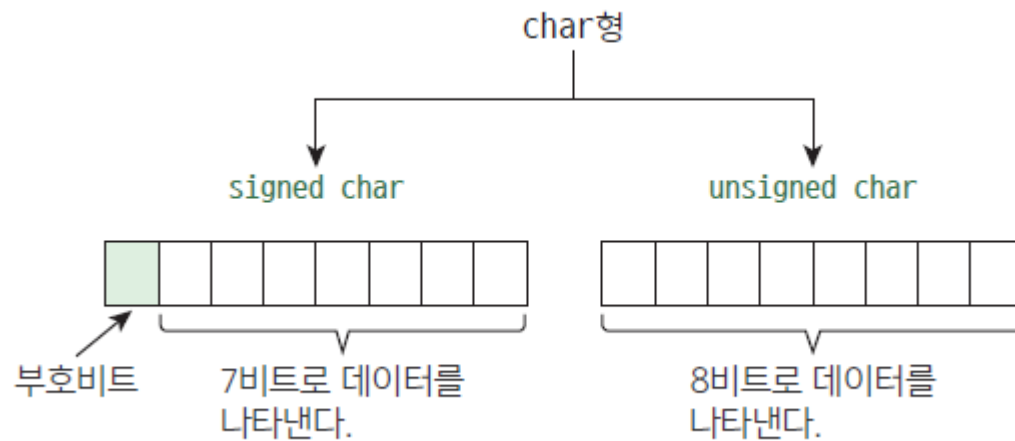
```
total_sale = 200000000
```



signed, unsigned 수식자

14

- unsigned
 - 음수가 아닌 값만을 나타냄을 의미
 - unsigned int
- signed
 - 부호를 가지는 값을 나타냄을 의미
 - 흔히 생략

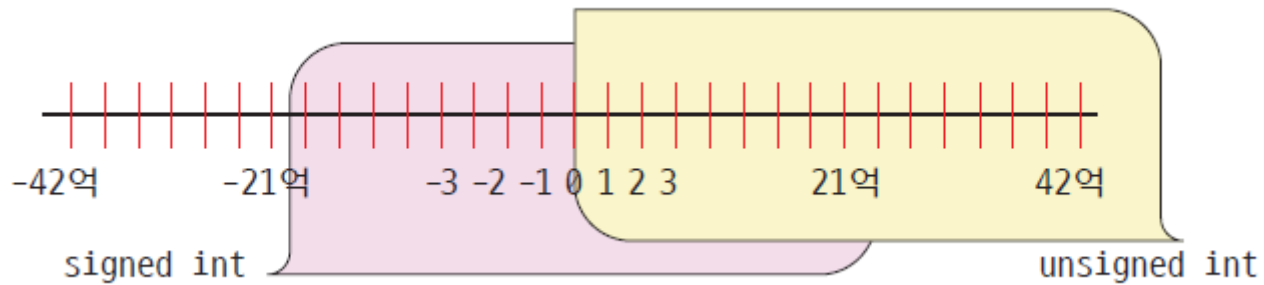




unsigned int

15

$0, 1, 2, \dots, 2^{32} - 1$
(0 ~ +4294967295)





unsigned 예제

16

```
unsigned int    speed;           // 부호없는 int형
unsigned        distance;        // unsigned int distance와 같다.
unsigned short  players;         // 부호없는 short형
```

```
unsigned int sales = 2800000000; // 약 28억
printf("%u \n", sales);          // %d를 사용하면 음수로 출력된다
```

unsigned는 %u로
출력하세요.





```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    short s_money = SHRT_MAX;           // 최대값으로 초기화한다. 32767
    unsigned short u_money = USHRT_MAX; // 최대값으로 초기화한다. 65535

    s_money = s_money + 1;
    printf("s_money = %d", s_money);

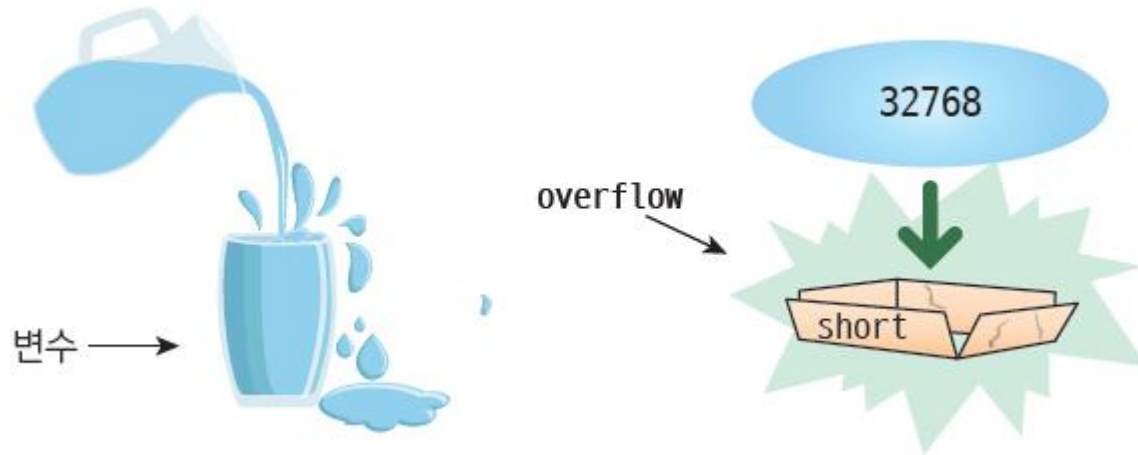
    u_money = u_money + 1;
    printf("u_money = %d", u_money);
    return 0;
}
```

오버플로우 발생!!

S_money = -32768
U_money = 0

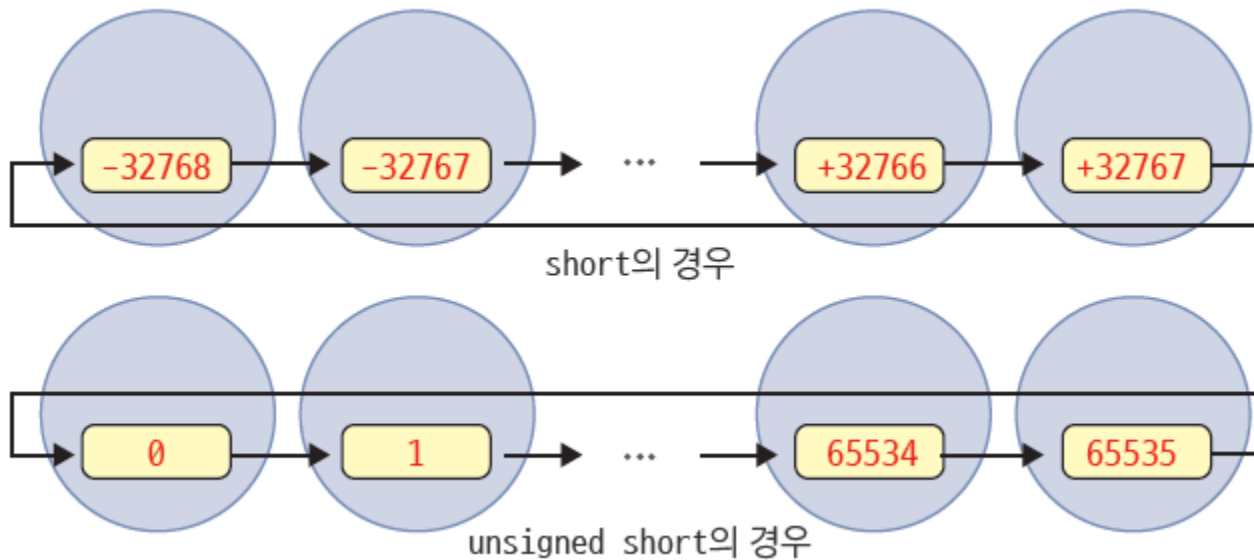


- **오버플로우(overflow)**: 변수가 나타낼 수 있는 범위를 넘는 숫자를 저장하려고 할 때 발생





- 규칙성이 있다.
 - 수도 계량기나 자동차의 주행거리계와 비슷하게 동작





참고사항

각 자료형의 최대값과 최소값은 limits.h에 정의되어 있다.

```
#define CHAR_MIN    (-128)
#define CHAR_MAX     127

#define SHRT_MIN     (-32768)      /* minimum (signed) short value */
#define SHRT_MAX     32767        /* maximum (signed) short value */
#define USHRT_MAX    0xffff       /* maximum unsigned short value */

#define INT_MIN      (-2147483647 - 1) /* minimum (signed) int value */
#define INT_MAX      2147483647      /* maximum (signed) int value */
#define UINT_MAX     0xffffffff      /* maximum unsigned int value */
```



10진법, 8진법, 16진법

21

- 8진법

- $012_8 = 1 \times 8^1 + 2 \times 8^0 = 10$

- 16진법

- $0xA_{16} = 10 \times 16^0 = 10$

10진수	8진수	16진수
0	00	0x0
1	01	0x1
2	02	0x2
3	03	0x3
4	04	0x4
5	05	0x5
6	06	0x6
7	07	0x7
8	010	0x8
9	011	0x9
10	012	0xa
11	013	0xb
12	014	0xc
13	015	0xd
14	016	0xe
15	017	0xf
16	020	0x10
17	021	0x11
18	022	0x12



예제

22

```
/* 정수 상수 프로그램*/  
#include <stdio.h>  
  
int main(void)  
{  
    int x = 10; // 10은 10진수이고 int형이고 값은 십진수로 10이다.  
    int y = 010; // 010은 8진수이고 int형이고 값은 십진수로 8이다.  
    int z = 0x10; // 010은 16진수이고 int형이고 값은 십진수로 16이다.  
  
    printf("x = %d", x);  
    printf("y = %d", y);  
    printf("z = %d", z);  
  
    return 0;  
}
```

x = 10
y = 8

x = 10
y = 8
z = 16



기호 상수

23

- 기호 상수(symbolic constant): 기호를 이용하여 상수를 표현한 것
- (예)
 - `won = 1120 * dollar;` // (1) 실제의 값을 사용
 - `won = EXCHANGE_RATE * dollar;` // (2) 기호상수 사용
- 기호 상수의 장점
 - 가독성이 높아진다.
 - 값을 쉽게 변경할 수 있다.



기호 상수의 장점

24

```
#include <stdio.h>

int main(void)
{
    ...
    won1 = 120 * dollar1;
    won2 = 120 * dollar2;
    ...
}
```

Diagram showing two green boxes labeled '1050' with red arrows pointing to the ~~120~~ values in the code, indicating that every occurrence of the literal must be updated.

리터럴 상수를 사용하는 경우:
등장하는 모든 곳을 수정하여야 한다.

```
#include <stdio.h>
#define EXCHANGE_RATE 120 1050

int main(void)
{
    ...
    won1 = EXCHANGE_RATE * dollar1;
    won2 = EXCHANGE_RATE * dollar2;
    ...
}
```

Diagram showing a green box labeled '1050' with a red arrow pointing to the ~~120~~ in the `#define` line, indicating that only the definition needs to be updated.

기호 상수를 사용하는 경우:
기호 상수가 정의된 곳만 수정하면 한다.



기호 상수를 만드는 방법 #1

25

Syntax

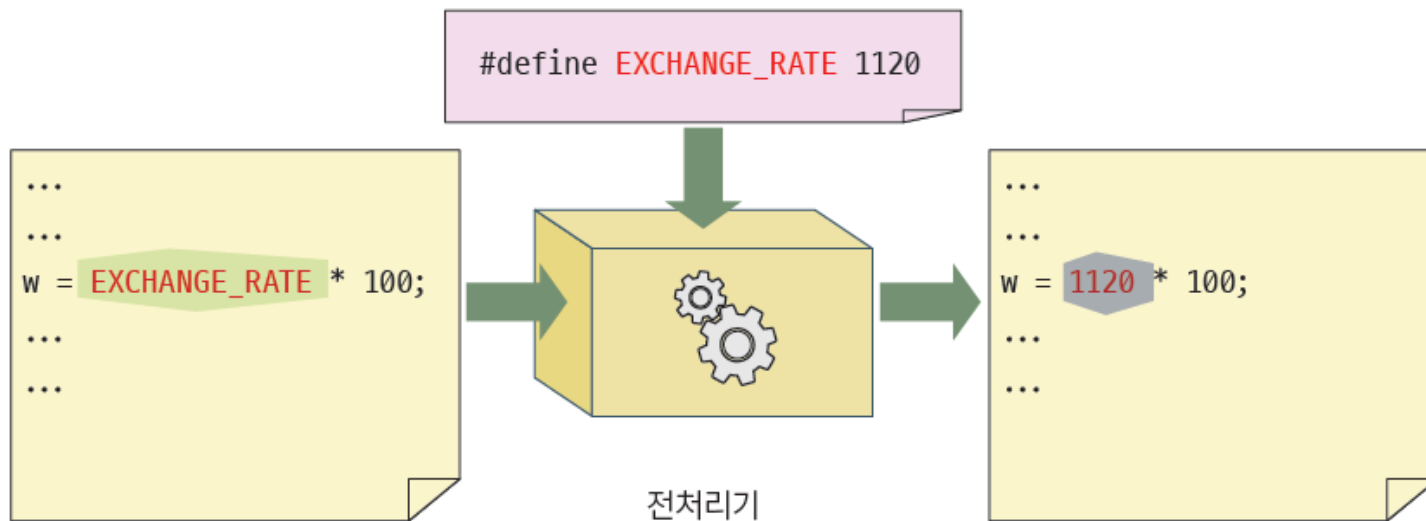
기호상수선언

예

```
#define EXCHANGE_RATE 1120
```

기호상수

값





기호 상수를 만드는 방법 #2

26

Syntax

기호상수선언

예

```
const int EXCHANGE_RATE = 1120;
```

기호상수

값



예제: 기호 상수

27

```
#include <stdio.h>
```

```
#define TAX_RATE 0.2
```

기호상수

```
int main(void)
```

```
{
```

```
    const int MONTHS = 12;
```

```
    int m_salary, y_salary;           // 변수 선언
```

```
    printf("월급을 입력하시요: "); // 입력 안내문
```

```
    scanf("%d", &m_salary);
```

```
    y_salary = MONTHS * m_salary;     // 순수입 계산
```

```
    printf("연봉은 %d입니다.", y_salary);
```

```
    printf("세금은 %f입니다.", y_salary*TAX_RATE);
```

```
    return 0;
```

```
}
```

월급을 입력하시요: 100
연봉은 1200입니다.
세금은 240.000000입니다.



정수 표현 방법

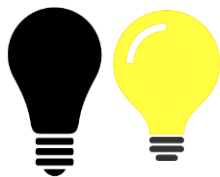
28

- 컴퓨터에서 정수는 이진수 형태로 표현되고 이진수는 전자 스위치로 표현된다.

스위치가 **하나** 있으면



0, 1



스위치가 **둘** 있으면



00,



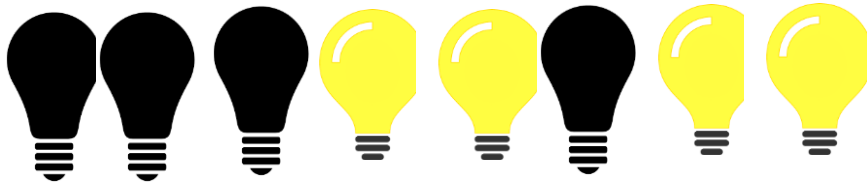
01,



10,



11





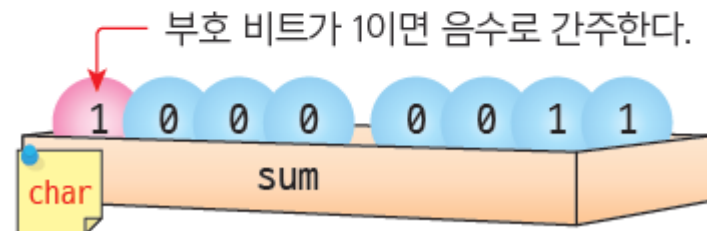
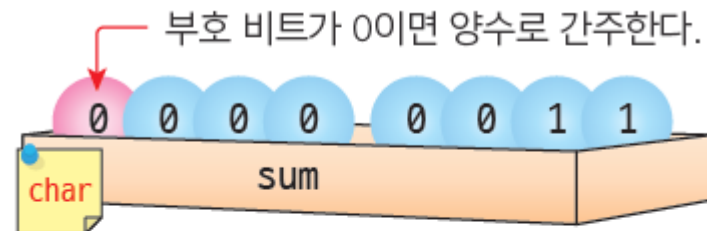
비트 패턴	정수	비고
0000000000000000	0	양의 정수
0000000000000001	1	
0000000000000010	2	
0000000000000011	3	
0000000000000100	4	
0000000000000101	5	
...	...	



정수 표현 방법

30

- 양수
 - 십진수를 이진수로 변환하여 저장하면 된다.
- 음수
 - 보통은 첫번째 비트를 부호 비트로 사용한다.
 - 단순 사용은 문제점이 발생한다.

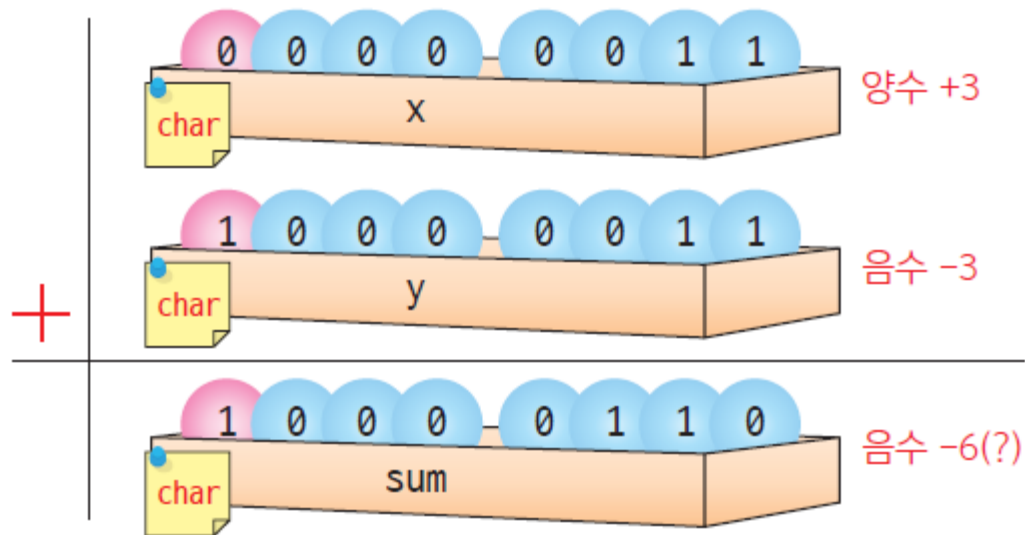




음수를 표현하는 첫번째 방법

31

- 첫번째 방법은 맨 처음 비트를 부호 비트로 간주하는 방법입니다.
- 양수와 음수의 덧셈 연산을 하였을 경우, 결과가 부정확하다.
 - (예) $+3 + (-3)$

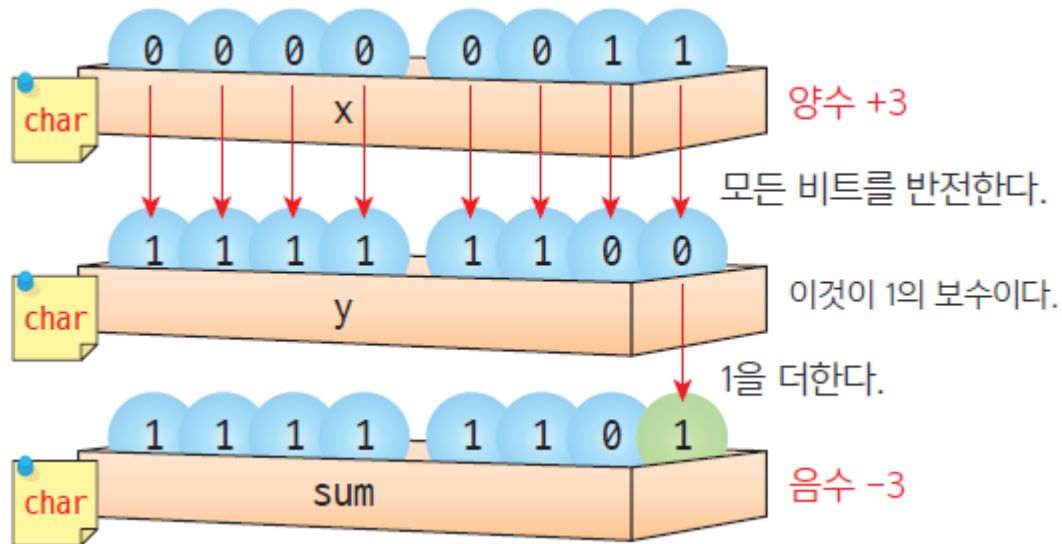




음수를 표현하는 두번째 방법

32

- 2의 보수로 음수를 표현한다. -> 표준적인 음수 표현 방법
- 2의 보수를 만드는 방법





2의 보수

33

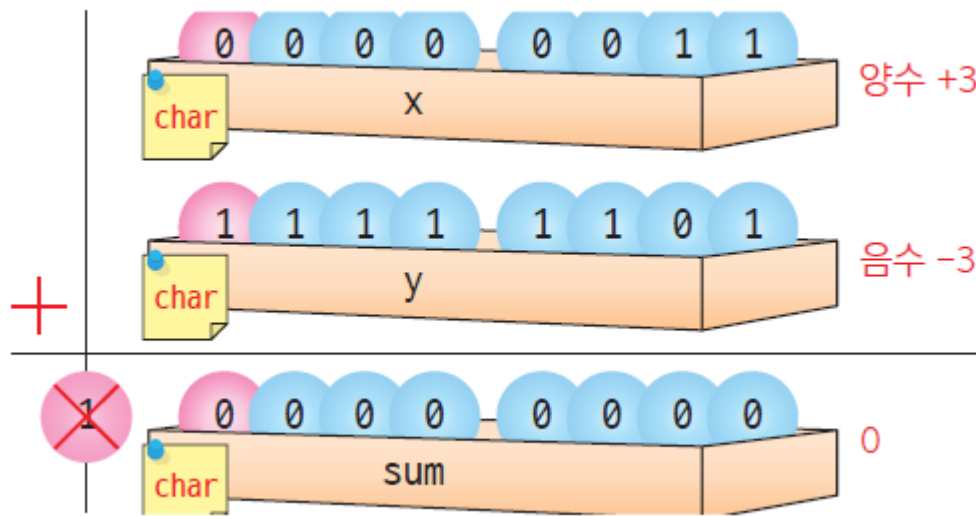
비트	부호없는 정수	부호있는 정수 (2의 보수)
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-4
101	5	-3
110	6	-2
111	7	-1

비트	부호없는 정수	부호있는 정수 (2의 보수)
0000 0000	0	0
0000 0001	1	1
0000 0010	2	2
0111 1110	126	126
0111 1111	127	127
1000 0000	128	-128
1000 0001	129	-127
1000 0010	130	-126
1111 1110	254	-2
1111 1111	255	-1



2의 보수로 양수와 음수를 더하면

34



음수를 2의 보수로 표현하면 양수와 음수를 더할 때 각각의 비트들을 더하면 됩니다.

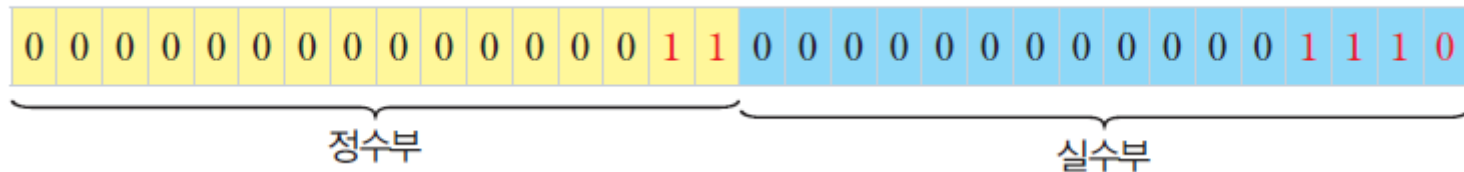




실수를 표현하는 방법 #1

35

- 정수 부분을 위하여 일정 비트를 할당하고 소수 부분을 위하여 일정 비트를 할당
- 전체가 32비트이면 정수 부분 16비트, 소수 부분 16비트 할당
- 과학과 공학에서 필요한 아주 큰 수를 표현할 수 없다



- 수학에서의 실수는 3.14와 같이 소수점을 가진 수이다. 실수는 매우 큰 수나 매우 작은 수를 다루는 과학이나 공학 분야의 응용 프로그램을 작성할 때는 없어서는 안 될 중요한 요소이다.

이 방법으로
나타낼 수 있는
최대수는
얼마일까요?





실수를 표현하는 방법 #2

36

● 부동 소수점 방식

부호비트(1비트)



s

e

지수부분(8비트)

m

가수부분(23비트)

$$149598000 = 1.49598 \times 10^8$$

$$\text{실수값} = (-1)^s * (1.m) * 2^{e-127}$$

한동안 아주 다양한
부동소수점 방법이
사 용 되 었 지 만
1985년부터는 IEEE
754 로 표 준 화
되었습니다.

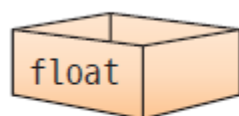
- 표현할 수 있는 범위가 대폭 늘어난다.
- 10^{-38} 에서 10^{+38}
- 정밀도는 유효 숫자(오차가 없는 숫자)의 개수로 나타내며,
가수부분에 몇 비트가 할당되느냐에 따라 결정된다.





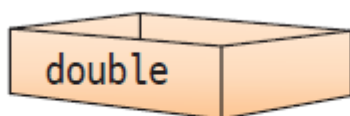
부동 소수점 형

37



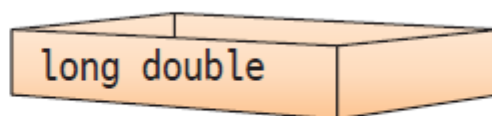
32비트

<=



64비트

<=



64비트

자료형	명칭	크기	범위
float	단일 정밀도(single-precision) 부동 소수점	32비트	$\pm 1.17549 \times 10^{-38} \sim \pm 3.40282 \times 10^{+38}$
double long double	두배 정밀도(double-precision) 부동 소수점	64비트	$\pm 2.22507 \times 10^{-308} \sim \pm 1.79769 \times 10^{+308}$



실수를 출력하는 형식 지정자

38

- %f
 - printf(“%f”, 0.123456789); // 0.123457 출력
- %e
 - printf(“%e”, 0.123456789); // 1.234568e-001 출력



예제

39

```
/* 부동 소수점 자료형의 크기 계산*/  
#include <stdio.h>  
int main(void)  
{  
    float x = 1.234567890123456789;  
    double y = 1.234567890123456789;  
  
    printf("float의 크기=%d\n", sizeof(float));  
    printf("double의 크기=%d\n", sizeof(double));  
  
    printf("x = %30.25f\n", x);  
    printf("y = %30.25f\n", y);  
    return 0;  
}
```

float의 크기=4

double의 크기=8

x = 1.23456788063049320000000000

y = 1.23456789012345670000000000



부동 소수점 상수

40

실수	지수 표기법	의미
123.45	1.2345e2	1.2345×10^2
12345.0	1.2345e4	1.2345×10^4
0.000023	2.3e-5	2.3×10^{-5}
2,000,000,000	2.0e9	2.0×10^9

1.23456

2.

.28

2e+10

9.26E3

0.67e-9

// 소수점만 붙여도 된다.

// 정수부가 없어도 된다.

// +나 -기호를 지수부에 붙일 수 있다.

// 9.26×10^3

// 0.67×10^{-9}



부동소수점 오버플로우

41

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
float x = 1e39;
```

```
printf("x = %e\n",x);
```

```
}
```

숫자가 커서 오버플로우
발생

x = inf

계속하려면 아무 키나 누르십시오 . . .



부동 소수점 언더플로우

42

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float x = 1.23456e-38;
```

```
    float y = 1.23456e-40;
```

```
    float z = 1.23456e-46;
```

```
    printf("x = %e\n",x);
```

```
    printf("y = %e\n",y);
```

```
    printf("z = %e\n",z);
```

```
}
```

숫자가 작아서
언더플로우 발생

```
x = 1.234560e-038
```

```
y = 1.234558e-040
```

```
z = 0.000000e+000
```



부동소수점형 사용시 주의사항

43

- 오차가 있을 수 있다!

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float value = 0.1;
```

```
    printf("%.20f \n", value);
```

```
    return 0;
```

```
}
```

이진법으로는 정확하게 나타낼 수 없는 값들이 있기 때문이다. 0.1도 그 중의 하나이다.

// %.20f는 소수점 이하를 20자리로 출력하라는 의미이다.

0.10000000149011611938



부동소수점 연산 사용시 주의사항

44

- 오차가 있을 수 있다!

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    double x;
```

```
    x = (1.0e20 + 5.0) - 1.0e20;
```

```
    printf("%f \n", x);
```

```
    return 0;
```

```
}
```

부동소수점 연산에서는
오차가 발생한다.
5.0이 아니라 0으로 계산
된다.

0.000000



문자형

45

- 문자는 컴퓨터보다는 인간에게 중요
- 문자도 숫자를 이용하여 표현
- 공통적인 규격이 필요하다.
- 아스키 코드(**ASCII**: American Standard Code for Information Interchange)





아스키 코드표 (일부)

46

Dec	Hex	문자
0	0	NULL
1	1	SOH
2	2	STX
3	3	ETX
4	4	EOL
5	5	ENQ
6	6	ACK
7	7	BEL
8	8	BS
9	9	HT
10	A	LF
11	B	VT
12	C	FF
13	D	CR
14	E	SO
15	F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3

Dec	Hex	문자
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	space
33	21	!
34	22	"
35	23	#
36	24	\$
37	25	%
38	26	&
39	27	'

Dec	Hex	문자
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;

Dec	Hex	문자
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O

[illegible]

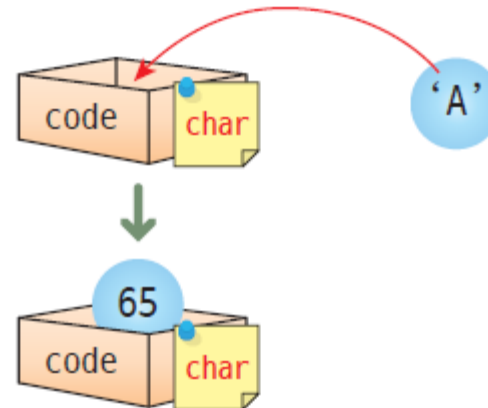


문자 변수

48

- char형을 사용하여 문자를 저장한다.

```
char code;  
code = 'A';
```





예제

49

```
/* 문자 변수와 문자 상수*/  
#include <stdio.h>  
  
int main(void)  
{  
    char code1 = 'A';    // 문자 상수로 초기화  
    char code2 = 65;     // 아스키 코드로 초기화  
  
    printf("code1 = %c\n", code1);  
    printf("code2 = %c\n", code2);  
}
```

```
code1 = A  
code2 = A
```



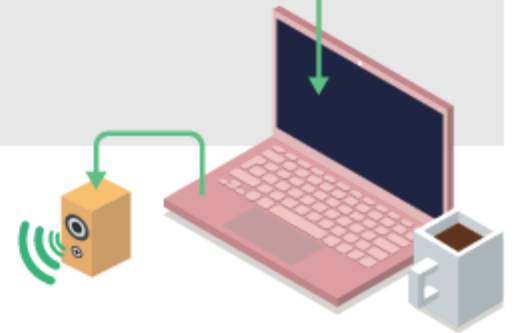
제어 문자

50

- 인쇄 목적이 아니라 제어 목적으로 사용되는 문자들
 - (예) 줄바꿈 문자, 탭 문자, 벨소리 문자, 백스페이스 문자

```
char beep = 7;  
printf("%c", beep);
```

```
printf("%c", 7);
```



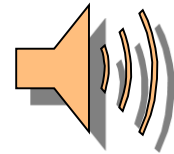


제어 문자를 나타내는 방법

51

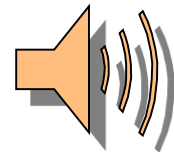
- 아스키 코드를 직접 사용

```
char beep = 7;  
printf("%c", beep);
```



- 이스케이프 시퀀스 사용

```
char beep = '\a';  
printf("%c", beep);
```





이스케이프 시퀀스(특수문자열)

52

제어 문자	이름	의미
\0	널문자	
\a	경고(bell)	"삐"하는 경고음 발생
\b	백스페이스(backspace)	커서를 현재의 위치에서 한 글자 뒤로 옮긴다.
\t	수평탭(horizontal tab)	커서의 위치를 현재 라인에서 설정된 다음 탭 위치로 옮긴다.
\n	줄바꿈(newline)	커서를 다음 라인의 시작 위치로 옮긴다.
\v	수직탭(vertical tab)	설정되어 있는 다음 수직 탭 위치로 커서를 이동
\f	폼피드(form feed)	주로 프린터에서 강제로 다음 페이지로 넘길 때 사용된다.
\r	캐리지 리턴(carriage return)	커서를 현재 라인의 시작 위치로 옮긴다.
\"	큰따옴표	원래의 큰따옴표 자체
\'	작은따옴표	원래의 작은따옴표 자체
\\	역슬래시(back slash)	원래의 역슬래시 자체



프로그램에서 경고음을 내려면?

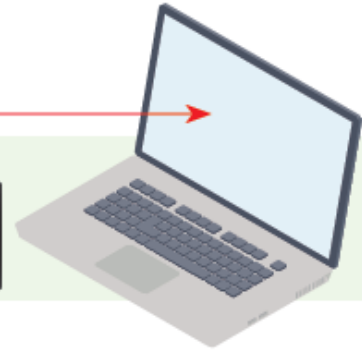
53

특수 문자열을 사용하여 프로그램에서 경고음을 내려면 다음과 한다.

```
char beep = '\a';  
printf("%c", beep);
```

```
printf("\a");
```

beep





역슬래시 \

54

- 특수한 기능을 가진 문자 앞에 역슬래시 \를 위치시키면 문자의 특수한 의미가 사라진다.

```
printf(" \"나만의 할리우드\" UCC 열풍 ");
```



“나만의 할리우드” UCC 열풍

```
printf(" \\는 제어 문자를 표시할 때 사용한다. ");
```



\\는 제어 문자를 표시할 때 사용한다.



정수형으로서의 char형

55

- 8비트의 정수를 저장하는데 char 형을 사용할 수 있다..

```
#include <stdio.h>

int main(void)
{
    char code = 'A';
    printf("%d %d %d \n", code, code + 1, code + 2); // 65 66 67이 출력된다.
    printf("%c %c %c \n", code, code + 1, code + 2); // A B C가 출력된다.
    return 0;
}
```



```
65 66 67
A B C
```

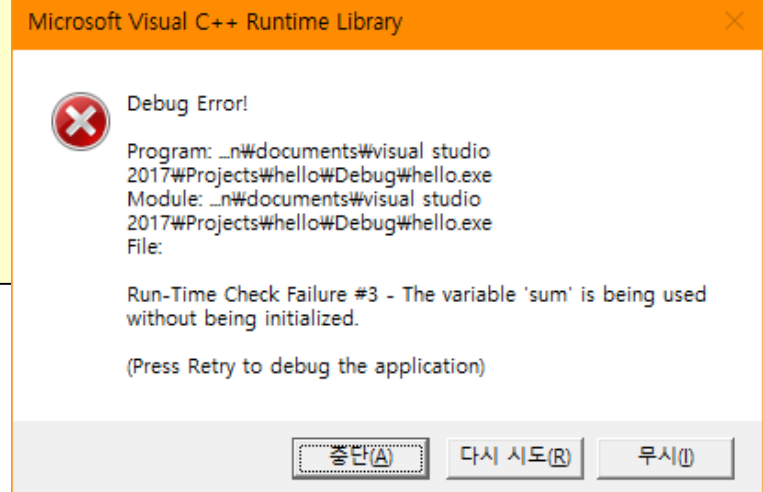


변수의 초기값

56

```
#include <stdio.h>
int main(void)
{
    int x, y, z, sum;

    printf("3개의 정수를 입력하세요 (x, y, z): ");
    scanf("%d %d %d", &x, &y, &z);
    sum += x;
    sum += y;
    sum += z;
    printf("3개 정수의 합은 %d\n", sum);
    return 0;
}
```





무엇이 문제일까?

57

```
#include <stdio.h>
int main(void)
{
    int x, y, z, sum;

    sum = 0;
    printf("3개의 정수를 입력하세요 (x, y, z): ");
    scanf("%d %d %d", &x, &y, &z);
    sum += x;
    sum += y;
    sum += z;
    printf("3개 정수의 합은 %d\n", sum);

    return 0;
}
```

변수는 사용하기 전에 반드시 초기화
시켜야 함!

3개의 정수를 입력하세요 (x, y, z): 10 20 30
3개 정수의 합은 60



Q & A

58

